

Universitat Autònoma de Barcelona
Master Degree in Modelling for Science and Engineering
Institution / Lab: IIIA-CSIC

TFM Architecture and Implementation Reference

ROS4HRI LLM Planner Stack

Document type:	Thesis architecture and implementation reference
Academic year:	2025-2026
Date:	2026-06-03
Scope:	Architecture, package ownership, runtime flow, and thesis chapter integration.
Repository:	https://github.com/ieverythng/nao-ros4hri-bridge

TFM Architecture and Implementation Reference

Purpose and Thesis Positioning

This document turns the current implementation into thesis-ready prose. It supports the architecture, implementation, and discussion chapters of the TFM draft extracted from the Overleaf structure. The system under study is a modular ROS 2 and ROS4HRI stack for the NAO robot in which natural language, symbolic grounding, LLM planning, deterministic execution, and user-facing dialogue are deliberately separated.

The core claim is that LLM-based planning becomes more inspectable and experimentally useful when the language model is not treated as a direct robot controller. Instead, the LLM is placed inside a typed planning layer. It receives bounded context, emits structured JSON, and depends on deterministic ROS nodes for admission, validation, execution, and feedback.

Architectural Thesis

The architecture follows a layered design. The dialogue layer is responsible for receiving user turns and producing semantic intent frames. The planning layer turns those intent frames into structured plans over an abstract skill registry. The orchestrator validates the plan and dispatches only supported actions. Skill servers interact with robot adapters, scene sources, and the knowledge base, then publish typed feedback that the planner can use for continuation, clarification, or replanning.

This split gives a concrete answer to the problem statement: an LLM planner can be integrated into a ROS4HRI robotic system by making it a supervised, contract-bound component rather than an unconstrained action generator. The planner decides what should happen at the symbolic level, while ROS components decide whether and how it may happen.

Runtime Ownership Model

packages	Responsibility	Thesis role
dialogue_manager`, `chatbot_llm`	Dialogue lifecycle, turn handling, intent declaration, user-facing language	Shows that conversational behavior is separate from planning
planner_llm`, `planner_common`	Plan generation, supervisor state, request/output/feedback contracts	Shows how LLM output is made structured and typed
orchestrator`, AB=1 skills	Request admission, plan validation, action dispatch, feedback	Shows deterministic control around the LLM
scene_grounding`, `kb_skills`, KnowledgeCore	Detector-to-KB grounding and prompt-facing state	Shows how the planner receives bounded symbolic context
robot_say_skill`, `nao_look_at`, motion/replay skills	Robot-specific execution details	Shows portability through abstract capability interfaces

End-to-End Runtime Flow

The runtime flow begins with a human utterance. The dialogue manager forwards the conversational turn to chatbot_llm, which classifies the turn as dialogue, knowledge query, or execution. Execution-eligible turns are converted into planner request JSON and published through the orchestrator planner gate. The

planner produces a plan, the orchestrator dispatches each step, and feedback is returned as structured JSON. User-facing status is emitted through planner dialogue acts and handled by the dialogue/speech owner.

Node and Package Responsibilities

`chatbot\_llm`

chatbot_llm is the user-facing LLM backend. Its role is not to generate robot plans directly. It interprets the user turn, creates a normalized intent frame, injects knowledge and scene context when available, and publishes planner requests when the turn requires execution. It also remains the appropriate owner for user-facing wording when planner dialogue completion is routed through chatbot-owned language generation.

`planner\_llm`

planner_llm is the high-level supervisor. It consumes admitted planner requests, reasons over the skill registry, and emits plan JSON. It tracks goal continuity through goal_id and plan lineage through plan_id and plan_version. It receives execution feedback and may continue, replan, ask for clarification, explain a failure, or emit completion dialogue.

`planner\_common`

planner_common is the contract source of truth. It normalizes planner requests, plan steps, execution feedback, scene summaries, and dialogue acts. This package is critical for thesis reproducibility because it makes the JSON interfaces explicit rather than leaving them implicit in prompt text.

`nao\_orchestrator`

nao_orchestrator is the deterministic execution boundary. It admits planner requests, validates plan steps, dispatches actions, and publishes feedback. It is intentionally not an LLM policy node. This is the main safety and reproducibility boundary: planner outputs become executable only after deterministic validation.

`nao\_scene\_grounding` and `kb\_skills`

nao_scene_grounding converts detector output into symbolic facts and scene summaries. kb_skills keeps KnowledgeCore access behind a package boundary. Together, they provide the grounding path that prevents the planner from relying only on language priors.

AB=1 skills and robot adapters

Skills such as say, look_at, scan, report_result, and head-motion actions form the execution vocabulary. They are represented to the planner as abstract capabilities but implemented through ROS actions and robot-specific adapters. This is the thesis bridge between symbolic planning and embodied robot behavior.

Architectural Invariants

- The planner emits structured plan JSON, not direct robot API calls.
- The orchestrator validates and dispatches; it does not generate LLM policy.
- The dialogue stack owns user-facing speech and conversational lifecycle.
- Scene and KB state are passed as bounded JSON context, not as raw detector or KB transport dumps.
- Execution feedback is the mechanism that connects embodied outcomes back to the planner.
- People and objects are represented separately in grounded context to avoid treating humans as generic objects.

Relation to ROS4HRI and SocialMinds Principles

The system follows ROS4HRI-style modularity by keeping public behavior in ROS messages, services, and actions. Continuous observations such as scene state are topic-like, short request-response operations remain service-like, and long-running robot behaviors are action-oriented. The architecture also preserves upstream-sensitive package boundaries: dialogue remains in dialogue packages, grounding remains in scene/KB packages, and robot execution remains in skill/adaptor packages.

For the thesis, this matters because the contribution is not simply that an LLM can produce a plausible task list. The contribution is that the LLM planner is embedded in a robotics architecture where responsibilities are inspectable and replaceable.

Chapter Integration Guidance

- Chapter 1: motivate the need for modular, grounded, feedback-aware LLM planning.
- Chapter 3: convert the invariants into design principles and non-functional requirements.
- Chapter 4: use the layered diagram and package responsibility table as the system architecture section.
- Chapter 6: expand the node descriptions into implementation subsections.
- Chapter 9: discuss why ownership boundaries reduce brittleness and improve observability.